

## Explainable Shortlist Assistant (ESA) - Feature

=====

Date: 2025-08-19

### 1. Executive Summary

-----

The Explainable Shortlist Assistant (ESA) is an employer-focused AI feature that ranks and shortlists candidates for a job posting, providing transparent, human-readable explanations and suggested interview questions. ESA helps employers quickly find the best-fit candidates, increases recruiter trust through explainability, and enables faster, more accurate hiring decisions using data already in the system.

### 2. Problem Statement

-----

Employers receive many applications and need an efficient, trustworthy way to surface top candidates. Current manual triage is time-consuming and inconsistent. Employers need concise reasons why a candidate is recommended and actionable interview prompts to evaluate candidates faster.

### 3. Proposed Solution

-----

ESA generates a ranked shortlist per job by combining semantic matching, skill coverage, experience weighting, and trust signals (e.g., reviews). For each candidate, ESA returns:

- A numeric `score` (0–100)
- `top\_matching\_skills`
- `explainable highlights` (2–4 bullets)
- `3 suggested interview questions`

Employers can accept, dismiss, or provide feedback on shortlisted candidates. Feedback is recorded to improve ranking calibration.

### 4. Key Benefits

- 
- Faster shortlist creation and reduced recruiter time-to-hire.
  - Explainable reasons for candidate ranking increase employer trust.
  - Actionable interview prompts tailored to candidate strengths/weaknesses.
  - Reuses existing data (applications, profiles, reviews) — minimal new data collection.

### 5. Components & Architecture

#### A. Data Sources

- 
- jobs (job descriptions, required skills)
  - applications (resume\_link, cover letter)
  - job\_seeker\_profiles (resume text, skills, experience\_years)

#### B. Core Components

- Resume text extraction (PDF → text)
- Scoring engine (composite score from semantic + skill coverage + experience + trust)
- Explainability module (template rules or LLM for bullets & questions)
- Cache table: job\_candidate\_scores to persist results
- Actions log: shortlist\_actions to log employer feedback

#### C. Data Flow

1. Extract resume text and normalize.
2. Generate or retrieve embeddings for job and resume.
3. Compute semantic similarity and skill coverage.
4. Produce composite score and short explanation.
5. Return results to employer UI and persist into job\_candidate\_scores.
6. Employer gives feedback → recorded in shortlist\_actions → used for calibration.

## 6. Schema

### A. job\_candidate\_scores

- id, job\_id, application\_id, candidate\_id, score, semantic\_score, skill\_coverage, experience\_score, trust\_signal, summary, questions, generated\_at

### B. ai\_briefs (optional)

- id, job\_id, application\_id, model, prompt\_hash, output, created\_at

### C. shortlist\_actions

- id, job\_id, application\_id, employer\_id, action (shortlist|dismiss|message), reason, created\_at

## 7. API Endpoints

### - POST /api/jobs/:jobId/ai/score

Request: optional application\_ids

Response: list of {application\_id, candidate\_id, score, summary, questions}

### - GET /api/jobs/:jobId/ai/shortlist

Returns cached or fresh scoring results.

### - POST /api/jobs/:jobId/ai/feedback

Body: {application\_id, action, employer\_id, note}

### - GET /api/jobs/:jobId/ai/metrics (optional)

Returns score distribution, counts, top signals.

## 8. Evaluation Metrics

- Precision@5: percentage of top-5 recommended who proceed to interview/hire.
- Employer acceptance rate of AI shortlists.
- Time-to-shortlist reduction.
- Candidate response rates after AI shortlist.

## Appendix: Example scoring formula (configurable)

$$\text{score} = 0.5 * \text{semantic\_sim} + 0.3 * \text{skill\_coverage} + 0.15 * \text{experience\_score} + 0.05 * \text{trust\_signal}$$

### Notes:

- skill\_coverage: fraction of required skills matched (0..1)
- experience\_score: normalized years of experience vs required